

FT. GROUP

Sistema Web de Gestión y División de Gastos Compartidos

Documento técnico de Arquitectura, Backlog, Esquema de Base de Datos y Documentación Técnica

Campo	Detalle
Proyecto	FT. GROUP
Versión del documento integrado	1.0 (fusión de v1.0 y v2.1)
Fecha de fusión	20 de septiembre de 2026
Backend	Node.js + Express + TypeScript + PostgreSQL
Frontend	React + Vite + React Router + Leaflet
Arquitectura	Hexagonal (Ports & Adapters)
Documentos fuente	1) Documento de Arquitectura, Backlog Inicial, Esquema de BD y Documentación Técnica (v1.0) — 2) FT_GROUP Esquema de BD y Documentación Técnica (v2.1)

Integrantes: Sanny Valery Agualimpia Rentería – María Ángela Roldán Pérez – Jheinson Jhamid Gutiérrez – Johan Martínez

Cliente: Ing. Jairo Coy

Universidad de San Buenaventura
Sede Bello

Tabla de contenido

1. Introducción 4

2. Alcance del sistema 4

- 2.1. Alcance implementado (versión actual entregada) 4
- 2.2. Alcance del diseño completo propuesto (7 módulos funcionales) 4

3. Documento de arquitectura 5

- 3.1. Arquitectura general 5
- 3.2. Estilo arquitectónico: arquitectura hexagonal (Ports & Adapters) 5
- 3.3. Arquitectura hexagonal del backend implementado 6
- 3.4. Dominio, casos de uso, puertos y adaptadores (diseño completo) 7
- 3.5. Arquitectura del frontend 8
- 3.6. Flujo de autenticación 8
- 3.7. Flujo arquitectónico propuesto: registro de un gasto 9
- 3.8. Seguridad 9
- 3.9. Atributos de calidad considerados 10
- 3.10. Justificación de la arquitectura seleccionada 10
- 3.11. Decisiones arquitectónicas iniciales 11
- 3.12. Consideraciones de implementación sugeridas 11
- 3.13. Estrategia de calidad y pruebas (ISO/IEC 29119, ISTQB, 25010, 12207) 11

4. Backlog inicial 12

- 4.1. Épicas 12
- 4.2. Historias de usuario 12
- 4.3. Criterios de aceptación 14

5. Esquema de base de datos 14

- 5.1. Modelo relacional implementado (versión actual) 14
- 5.2. Modelo entidad-relación completo (diseño propuesto) 15
- 5.3. Diccionario de datos completo (DDL lógico) 16
- 5.4. Relaciones e índices 19

6. Documentación técnica 19

- 6.1. Tecnologías 19
- 6.2. Estructura de carpetas 20
- 6.3. API REST (implementada) 20
- 6.4. Rutas del frontend 20
- 6.5. Variables de entorno 20
- 6.6. Instalación y ejecución 21
- 6.7. Pruebas 21

7. Estado actual, limitaciones y trabajo futuro 21

8. Notas y limitaciones de la fusión de documentos 22

9. Conclusiones 22

1. Introducción

FT. GROUP es una aplicación web orientada a la gestión de usuarios y a la gestión y división de gastos compartidos. El proyecto cuenta con dos niveles de documentación que se integran en este documento único:

- Una implementación actual entregada, construida con un backend en Node.js, Express y TypeScript bajo arquitectura hexagonal (Ports and Adapters), y un frontend en React y Vite, que cubre el módulo de usuarios y autenticación.
- Una propuesta de diseño técnico ampliado, derivada del anteproyecto académico "FT. GROUP (Sistema Web de Gestión y División de Gastos Compartidos)" — Universidad de San Buenaventura, agosto de 2026 — que describe los 7 módulos funcionales completos del sistema (autenticación, grupos, gastos, motor de cálculo, historial, panel de usuario y escaneo inteligente de recibos).

El presente documento fusiona ambas fuentes: se conserva el nivel de detalle de la implementación real ya construida (endpoints, tablas efectivamente creadas, variables de entorno, instalación, pruebas) y se complementa con el diseño completo de base de datos y arquitectura para los módulos aún no implementados (grupos, gastos, participaciones, balances, movimientos, notificaciones y escaneo de recibos), de modo que cada documento aporta lo que al otro le faltaba.

El backend aplica una arquitectura hexagonal, separando las reglas del dominio de los detalles de infraestructura. El módulo implementado permite registrar usuarios, iniciar y cerrar sesión, renovar la sesión, consultar y actualizar el perfil y desactivar o activar una cuenta. El frontend proporciona una interfaz inicial funcional con registro, login, rutas protegidas, dashboard, perfil y un mapa basado en Leaflet/OpenStreetMap.

2. Alcance del sistema

2.1. Alcance implementado (versión actual entregada)

- Gestión de usuarios mediante registro y consulta del perfil autenticado.
- Autenticación mediante access token JWT y refresh token persistido en PostgreSQL.
- Actualización de nombre y teléfono del usuario.
- Desactivación lógica (soft-delete) y activación de la propia cuenta.
- Persistencia de usuarios y sesiones mediante PostgreSQL.
- Validación de datos de entrada mediante Zod.
- Protección básica de cabeceras HTTP con Helmet, CORS y consultas SQL parametrizadas.
- Interfaz inicial en React con login, registro, dashboard y perfil.
- Mapa funcional con Leaflet/OpenStreetMap y marcadores de demostración.

Nota de alcance: aunque el requisito general del proyecto menciona un CRUD de usuarios, la implementación actual está enfocada al ciclo de vida del propio usuario autenticado. No existe todavía un endpoint administrativo para listar todos los usuarios ni un borrado físico. El DELETE disponible corresponde a una desactivación lógica de la propia cuenta.

2.2. Alcance del diseño completo propuesto (7 módulos funcionales)

El diseño técnico ampliado cubre los 7 módulos funcionales descritos en la propuesta original del proyecto, de los cuales el módulo de autenticación es el único implementado hasta la fecha:

Módulo	Entidades de datos asociadas	Estado
Autenticación	USUARIO, REFRESH_TOKEN, PASSWORD_RESET_TOKEN	Parcialmente implementado (falta password reset)

Módulo	Entidades de datos asociadas	Estado
Gestión de grupos	GRUPO, MIEMBRO_GRUPO	Pendiente
Registro de gastos	GASTO	Pendiente
Motor de cálculo	PARTICIPACION_GASTO, BALANCE	Pendiente
Historial financiero	MOVIMIENTO	Pendiente
Panel de usuario	NOTIFICACION (+ vistas sobre BALANCE/GASTO)	Parcialmente implementado (dashboard con datos demo)
Escaneo inteligente de recibos (Deep Learning)	RECIBO_ESCANEADO	Pendiente

Las entidades, atributos y relaciones del diseño ampliado son una propuesta técnica inferida a partir de los objetivos, módulos y arquitectura descritos en el documento original del anteproyecto (autenticación JWT, gestión de grupos, motor de cálculo, historial, panel de usuario, escaneo de recibos por Deep Learning y reportes), y fueron unificadas con la nomenclatura de la implementación real donde ambas coinciden.

3. Documento de arquitectura

3.1. Arquitectura general

La solución está organizada en dos aplicaciones que se comunican mediante una API REST. El navegador ejecuta el frontend React y realiza peticiones HTTP al backend Express. El backend procesa autenticación y operaciones de usuario y utiliza PostgreSQL como sistema de persistencia.

Capa	Componente	Función
Presentación	React + Vite	Login, registro, dashboard, perfil y navegación.
API / Entrada	Express + Controllers + Routes	Recibir peticiones, validar entradas y devolver respuestas HTTP.
Aplicación / Dominio	Use Cases + Entidad Usuario + Ports	Coordinar reglas de negocio y desacoplar infraestructura.
Infraestructura	PostgreSQL + JWT + Bcrypt	Persistencia, hash de contraseñas, tokens y conexión a BD.

Flujo simplificado:

```

Usuario → Frontend React (Vite) → [HTTP/JSON] → Express / API REST → (Validadores Zod,
Controladores, Middleware JWT) → Casos de uso → Puertos (interfaces) → UsuarioRepository →
PostgreSQL/USUARIO · RefreshTokenRepository → PostgreSQL/REFRESH_TOKEN · PasswordHasher →
Bcrypt · TokenService → JWT + crypto

```

3.2. Estilo arquitectónico: arquitectura hexagonal (Ports & Adapters)

FT. GROUP adopta la Arquitectura Hexagonal (Ports and Adapters) como estilo arquitectónico oficial, en reemplazo del enfoque Microkernel considerado preliminarmente en el anteproyecto. Su principio fundamental consiste en ubicar las reglas de negocio en el centro del sistema (el núcleo) y establecer mecanismos de comunicación con el exterior mediante puertos, con los que interactúan los componentes externos a través de adaptadores. De esta manera, el núcleo no depende directamente de una interfaz gráfica, una base de datos o un servicio externo específico.

- Puertos de entrada: permiten que elementos externos soliciten la ejecución de funcionalidades de la aplicación.

- Puertos de salida: definen las operaciones que el núcleo necesita realizar sobre componentes externos.
- Adaptadores de entrada: traducen las solicitudes externas al formato que comprende la aplicación (p. ej. la API REST).
- Adaptadores de salida: implementan las interfaces necesarias para comunicarse con infraestructura externa (persistencia, servicios externos).

Objetivos de la arquitectura: separar las reglas de negocio de la interfaz de usuario; mantener el dominio independiente de la tecnología de almacenamiento; permitir incorporar o sustituir adaptadores externos sin modificar el núcleo; facilitar las pruebas de las reglas de negocio; mejorar la mantenibilidad y evolución del sistema.

FT. GROUP — Arquitectura Hexagonal (Ports & Adapters)

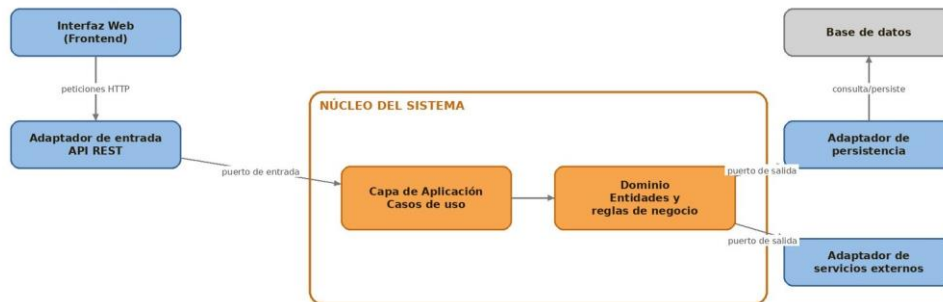


Figura 1. Arquitectura Hexagonal (Ports & Adapters) de FT. GROUP.

3.3. Arquitectura hexagonal del backend implementado

La arquitectura hexagonal permite que el núcleo de la aplicación no dependa directamente de Express, PostgreSQL o librerías concretas. Las interfaces o puertos definen qué necesita el dominio y los adaptadores implementan esas necesidades. Esta tabla refleja la estructura de carpetas actualmente implementada:

Capa	Carpeta / elemento	Responsabilidad
Dominio	domain/entities/Usuario.ts	Entidad Usuario, estados y reglas básicas.
Dominio	domain/ports/in	Casos de uso expuestos por el sistema.
Dominio	domain/ports/out	Contratos para persistencia, contraseñas y tokens.
Aplicación	application/usecases	Implementación de registrar, login, refresh, logout, perfil y estado.
Entrada	infrastructure/.../routes	Definición de endpoints HTTP.
Entrada	infrastructure/.../controllers	Traducción entre HTTP y casos de uso.
Entrada	middlewares	Autenticación, validación y manejo de errores.
Salida	persistence/postgres	Implementación de repositorios con PostgreSQL.

Capa	Carpeta / elemento	Responsabilidad
Salida	security	Implementación de Bcrypt y JWT.
Configuración	infrastructure/config	Variables de entorno, contenedor y migraciones.

3.4. Dominio, casos de uso, puertos y adaptadores (diseño completo)

El dominio constituye el núcleo conceptual del sistema; contiene las entidades y reglas relacionadas directamente con la gestión de gastos compartidos, incluyendo tanto lo ya implementado (Usuario) como lo proyectado para los módulos siguientes:

Entidad de dominio	Descripción	Estado
Usuario	Representa a una persona que utiliza el sistema y participa en uno o varios grupos.	Implementada
Grupo	Representa un espacio compartido en el que varios usuarios gestionan gastos.	Diseñada, pendiente de implementar
Gasto	Representa una obligación económica registrada dentro de un grupo.	Diseñada, pendiente de implementar
Participación	Representa la parte del gasto que corresponde a un integrante.	Diseñada, pendiente de implementar
Balance	Representa el estado de las obligaciones económicas entre los integrantes de un grupo.	Diseñada, pendiente de implementar
Movimiento	Permite registrar las operaciones relevantes realizadas sobre los gastos y balances.	Diseñada, pendiente de implementar

Nota de consistencia: estas entidades del dominio se materializan en persistencia como USUARIO, GRUPO, MIEMBRO_GRUPO, GASTO, PARTICIPACION_GASTO, BALANCE y MOVIMIENTO — nomenclatura unificada con el diccionario de datos de la sección 5 de este documento.

3.4.1. Casos de uso identificados

- Registrar usuario. (implementado)
- Iniciar sesión. (implementado)
- Renovar sesión, cerrar sesión, consultar/actualizar perfil, activar/desactivar cuenta. (implementado)
- Crear un grupo. (diseñado, pendiente)
- Administrar integrantes de un grupo. (diseñado, pendiente)
- Registrar un gasto. (diseñado, pendiente)
- Dividir un gasto entre integrantes. (diseñado, pendiente)
- Consultar balances. (diseñado, pendiente)
- Consultar historial de movimientos. (diseñado, pendiente)

3.4.2. Puertos de entrada

- Puerto de autenticación.
- Puerto de gestión de usuarios.
- Puerto de gestión de grupos.
- Puerto de gestión de gastos.
- Puerto de división de gastos.
- Puerto de consulta de balances.

- Puerto de consulta de historial.

3.4.3. Puertos de salida

El principal puerto de salida es el de persistencia de información. También se contempla un puerto para servicios externos, orientado a funcionalidades complementarias que puedan incorporarse posteriormente (por ejemplo, el módulo de escaneo de recibos).

3.4.4. Adaptadores

- Adaptador de entrada (API REST): recibe las solicitudes del frontend, valida los datos de entrada y las transforma en solicitudes compatibles con los casos de uso, evitando que el dominio conozca protocolos HTTP o detalles del frontend.
- Adaptador de persistencia: implementa los puertos definidos por la aplicación para almacenar y recuperar información, gestionando la comunicación con la base de datos sin introducir dependencias de infraestructura en las reglas de negocio.
- Adaptador de servicios externos: permite incorporar servicios externos cuando sean necesarios para el funcionamiento o ampliación de FT. GROUP (su implementación dependerá del servicio concreto que se integre, p. ej. el módulo de escaneo de recibos).

3.5. Arquitectura del frontend

El frontend utiliza React con Vite. React Router administra la navegación y las rutas protegidas. El archivo api.js centraliza las llamadas HTTP al backend y administra los tokens en sessionStorage para mantener compatibilidad con el backend actual.

Componente	Responsabilidad
App.jsx	Define rutas, sesión inicial y protección de páginas.
api.js	Centraliza fetch, login, registro, refresh, perfil, logout y estado de cuenta.
Login.jsx	Formulario de autenticación.
Register.jsx	Formulario de creación de cuenta.
Dashboard.jsx	Indicadores iniciales, actividad y mapa.
Profile.jsx	Consulta, actualización y desactivación de cuenta.
Layout.jsx	Barra lateral, navegación y encabezado.
MapView.jsx	Mapa Leaflet con marcadores de ejemplo.

3.6. Flujo de autenticación

1. El usuario registra una cuenta desde /registro.
2. El backend valida los datos, verifica que el correo no exista y genera un hash de la contraseña con Bcrypt.
3. Al iniciar sesión, el backend compara la contraseña con el hash almacenado.
4. Si el usuario está activo, se genera un access token JWT con duración configurada (por defecto 15 minutos).
5. También se genera un refresh token aleatorio de mayor duración, almacenado en la tabla REFRESH_TOKEN.
6. El frontend guarda ambos tokens en sessionStorage en la implementación actual.
7. Las rutas protegidas envían el access token mediante Authorization: Bearer <token>.
8. Cuando se necesita renovar la sesión, /api/auth/refresh revoca el refresh token usado y genera uno nuevo.
9. Logout revoca el refresh token activo y el frontend elimina los tokens locales.

3.7. Flujo arquitectónico propuesto: registro de un gasto

Flujo general para el registro de un gasto, aplicable una vez se implemente el módulo de gastos siguiendo la misma arquitectura hexagonal ya usada para autenticación: (1) el usuario introduce la información desde el frontend; (2) el frontend envía la solicitud a la API REST; (3) el adaptador de entrada valida y transforma la solicitud; (4) la solicitud llega al caso de uso correspondiente en la capa de aplicación; (5) la aplicación aplica las reglas del dominio para procesar el gasto; (6) el sistema determina las participaciones de cada integrante; (7) la aplicación usa el puerto de persistencia para almacenar la información; (8) el adaptador de persistencia ejecuta la operación en la base de datos; (9) el resultado se retorna hacia la aplicación y el frontend.

Flujo: registro de un gasto

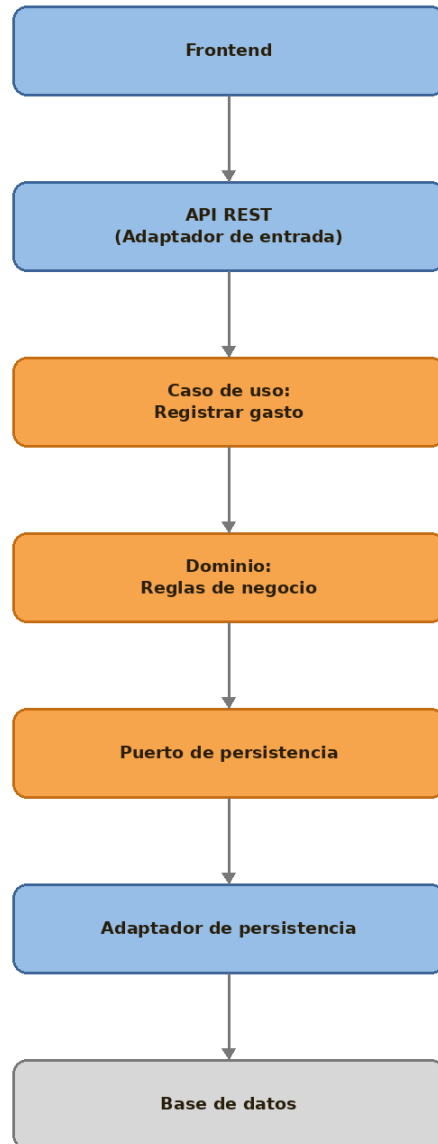


Figura 2. Flujo arquitectónico propuesto para el registro de un gasto.

3.8. Seguridad

Medidas de seguridad efectivamente implementadas en la versión actual:

- Bcrypt para almacenamiento de contraseñas mediante hash; nunca se almacena la contraseña en texto plano.
- JWT para access tokens de corta duración.
- Refresh tokens opacos generados con `crypto.randomBytes` y almacenados en base de datos.
- Rotación de refresh token: el token utilizado para renovar la sesión se revoca.
- Helmet para cabeceras HTTP de seguridad.
- CORS configurable mediante `CORS_ORIGIN`.
- Validación de entradas con Zod.
- Consultas PostgreSQL parametrizadas para reducir riesgo de inyección SQL.
- Manejo centralizado de errores sin exponer detalles internos inesperados.

Consideración importante: en la versión actual, el refresh token se entrega en JSON y el frontend lo conserva en `sessionStorage`. El requisito de cookies seguras (`HttpOnly`, `Secure` y `SameSite`) todavía no está implementado. Para una versión de producción se recomienda migrar el refresh token a una cookie `HttpOnly/Secure` y configurar CORS con credenciales.

Mecanismos de seguridad contemplados en el diseño ampliado para los módulos siguientes:

- Autorización basada en el usuario y su pertenencia a los grupos.
- Sanitización de entradas provenientes del usuario.
- Protección frente a ataques XSS, además de la ya cubierta inyección SQL.
- Control de expiración y validación de tokens en todos los módulos nuevos.

La separación proporcionada por la arquitectura hexagonal permite mantener estas responsabilidades distribuidas de manera controlada, evitando que las reglas de negocio dependan directamente de los mecanismos específicos de comunicación externa.

3.9. Atributos de calidad considerados

Atributo	Consideración arquitectónica
Seguridad	Separación de responsabilidades, autenticación, autorización, validación de entradas y manejo seguro de credenciales.
Mantenibilidad	Separación entre dominio, aplicación e infraestructura para facilitar modificaciones.
Escalabilidad	Posibilidad de incorporar nuevos adaptadores y servicios sin modificar directamente las reglas del dominio.
Confiabilidad	Centralización de las reglas de negocio para reducir inconsistencias en el procesamiento de gastos.
Eficiencia	Procesamiento organizado de las operaciones y acceso controlado a los mecanismos de persistencia.
Usabilidad	La interfaz web se mantiene separada de la lógica interna, permitiendo evolucionar la experiencia del usuario independientemente.

3.10. Justificación de la arquitectura seleccionada

La selección de la arquitectura hexagonal para FT. GROUP responde a la necesidad de mantener separadas las reglas de negocio de los componentes externos. Sus principales ventajas para el proyecto son:

- Independencia tecnológica: el dominio no depende directamente de una tecnología específica de base de datos, interfaz o servicio externo.

- **Mantenibilidad:** los cambios realizados en infraestructura pueden mantenerse aislados del núcleo del sistema.
- **Testabilidad:** los casos de uso y reglas de negocio pueden probarse sin depender necesariamente de componentes externos.
- **Flexibilidad:** los adaptadores pueden ser reemplazados o modificados manteniendo los contratos definidos mediante los puertos.
- **Separación de responsabilidades:** cada parte del sistema posee una función claramente definida.

Esta organización resulta adecuada para FT. GROUP debido a que el sistema debe manejar reglas de negocio relacionadas con la distribución de gastos y los balances entre usuarios, las cuales deben mantenerse independientes de la infraestructura tecnológica.

3.11. Decisiones arquitectónicas iniciales

Decisión	Justificación
Arquitectura Hexagonal	Permite separar el núcleo del sistema de los componentes externos.
Dominio independiente	Las reglas de negocio de FT. GROUP no deben depender de tecnologías de infraestructura.
API como adaptador de entrada	Permite recibir solicitudes del frontend y comunicarlás con los casos de uso.
Persistencia mediante adaptador	Evita que el dominio dependa directamente de la base de datos.
Uso de puertos	Define contratos claros entre el núcleo y los adaptadores.
Seguridad desde la arquitectura	Permite incorporar autenticación, autorización y validación desde las primeras etapas.

3.12. Consideraciones de implementación sugeridas

- **Backend:** framework compatible con JWT y transacciones ACID, organizado según los principios de puertos y adaptadores (p. ej. Spring Boot, NestJS o Express + un ORM transaccional, con el dominio aislado del framework). La implementación actual ya usa Express + TypeScript siguiendo este principio.
- **Base de datos:** motor relacional (PostgreSQL o MySQL) por la necesidad de integridad referencial, transacciones ACID y precisión decimal (DECIMAL, no FLOAT, para montos), accedido únicamente a través del adaptador de persistencia. La implementación actual usa PostgreSQL 16.
- **Servicios externos** (p. ej. escaneo de recibos): integrados mediante un adaptador de servicios externos independiente, sin acoplar el dominio a su implementación concreta.
- **Frontend:** aplicación responsiva (web/móvil) que consume la API REST (adaptador de entrada) mediante los contratos definidos por los puertos. La implementación actual usa React + Vite.

3.13. Estrategia de calidad y pruebas (ISO/IEC 29119, ISTQB, ISO/IEC 25010, ISO/IEC 12207)

La arquitectura hexagonal favorece la testabilidad al permitir probar los casos de uso y las reglas de negocio del dominio sin depender de componentes externos (base de datos, API, servicios). Sobre esa base se aplica el siguiente esquema de pruebas, incluyendo su aplicación en un futuro módulo de inteligencia artificial para escaneo de recibos:

Tipo de prueba	Aplicación general	Aplicación en módulo IA (recibos)
Unitarias	Funciones del núcleo y módulos (validaciones, cálculos)	Preprocesamiento de imagen, parsing de monto/fecha
Integración	Comunicación núcleo ↔ módulos vía API	Backend ↔ microservicio de IA (imagen → JSON)

Tipo de prueba	Aplicación general	Aplicación en módulo IA (recibos)
Funcionales	Casos de uso de cada módulo	Extracción correcta sobre recibos de prueba conocidos
No funcionales	Tiempos de respuesta < 2s por petición	Inferencia < 2-3s por imagen
Robustez / valores límite	Entradas inválidas, concurrencia	Imágenes borrosas, rotadas, manuscritas, formatos no vistos
Regresión	Tras cada cambio de código	Tras cada reentrenamiento del modelo
Aceptación (UAT), sistema, humo	Cobertura de todo el ciclo de vida (ISO/IEC 12207)	—

Cobertura de pruebas automatizadas objetivo: > 80 %.

Pruebas actualmente existentes en el repositorio (Jest, backend de usuarios y autenticación):

Archivo	Cobertura principal
RegistrarUsuarioUseCase.test.ts	Registro exitoso, correo duplicado y contraseña demasiado corta.
IniciarSesionUseCase.test.ts	Login correcto, usuario inexistente, contraseña incorrecta y usuario bloqueado.

4. Backlog inicial

El backlog se construye a partir de las funcionalidades implementadas y de las funcionalidades necesarias para completar progresivamente el sistema de gestión de gastos compartidos. Las prioridades son orientativas para planificación y no representan una evaluación del producto.

4.1. Épicas

ID	Épica	Descripción
E1	Gestión de usuarios	Registro, consulta, actualización y estado de cuentas.
E2	Autenticación y seguridad	Login, JWT, refresh, logout y protección de rutas.
E3	Dashboard	Indicadores y resumen de la actividad.
E4	Grupos	Creación y administración de grupos de gastos.
E5	Gastos compartidos	Registro, distribución, pagos y pendientes.
E6	Ubicación / mapa	Visualización de actividades o lugares asociados.
E7	Administración	Gestión administrativa de usuarios y monitoreo.

4.2. Historias de usuario

ID	Épica	Historia de usuario	Prioridad	Estado
HU-01	E1	Como visitante, quiero registrarme para crear una cuenta.	Alta	Implementada
HU-02	E2	Como usuario, quiero iniciar sesión con correo y contraseña.	Alta	Implementada
HU-03	E2	Como usuario, quiero mantener mi sesión mediante refresh token.	Alta	Implementada

ID	Épica	Historia de usuario	Prioridad	Estado
HU-04	E2	Como usuario, quiero cerrar sesión y revocar mi sesión.	Alta	Implementada
HU-05	E1	Como usuario, quiero consultar mi perfil.	Alta	Implementada
HU-06	E1	Como usuario, quiero actualizar nombre y teléfono.	Alta	Implementada
HU-07	E1	Como usuario, quiero desactivar mi cuenta.	Media	Implementada
HU-08	E1	Como usuario, quiero reactivar mi cuenta.	Media	Implementada en API
HU-09	E3	Como usuario, quiero ver un dashboard con indicadores básicos.	Alta	Implementada
HU-10	E6	Como usuario, quiero ver un mapa de actividades.	Media	Implementada con datos demo
HU-11	E4	Como usuario, quiero crear y administrar grupos.	Alta	Pendiente
HU-12	E5	Como usuario, quiero registrar un gasto compartido.	Alta	Pendiente
HU-13	E5	Como usuario, quiero dividir un gasto entre integrantes.	Alta	Pendiente
HU-14	E5	Como usuario, quiero consultar gastos pendientes y pagados.	Alta	Pendiente
HU-15	E7	Como administrador, quiero listar usuarios.	Media	Pendiente
HU-16	E7	Como administrador, quiero actualizar el estado de un usuario.	Media	Pendiente
HU-17	E7	Como administrador, quiero desactivar usuarios mediante soft-delete.	Media	Pendiente
HU-18	E2	Como usuario, quiero que el refresh token se maneje mediante cookie segura.	Alta	Pendiente
HU-19	E1	Como usuario, quiero poder recuperar mi contraseña mediante un enlace de un solo uso.	Alta	Pendiente (diseño de datos disponible: PASSWORD_RESET_TOKEN)
HU-20	E5	Como usuario, quiero escanear un recibo con la cámara para registrar el gasto automáticamente.	Media	Pendiente (diseño de datos disponible: RECIBO_ESCANEADO)
HU-21	E3	Como usuario, quiero recibir notificaciones de nuevos gastos, pagos y recordatorios.	Media	Pendiente (diseño de datos disponible: NOTIFICACION)
HU-22	E5	Como usuario, quiero consultar el historial de movimientos de un grupo con fines de auditoría.	Media	Pendiente (diseño de datos disponible: MOVIMIENTO)

Las historias HU-19 a HU-22 se incorporan a partir del diseño de base de datos ampliado (documento fuente 2), que ya definía las entidades PASSWORD_RESET_TOKEN, RECIBO_ESCANEADO, NOTIFICACION y MOVIMIENTO sin que existieran historias de usuario explícitas asociadas en el backlog original; se agregan aquí para dejar trazabilidad completa entre el modelo de datos y el backlog.

4.3. Criterios de aceptación

Historia	Criterio de aceptación
HU-01	Con datos válidos se crea el usuario; con correo repetido se responde 409; la contraseña se almacena como hash.
HU-02	Con credenciales válidas se devuelve accessToken, refreshToken y usuario; con credenciales inválidas se responde 401.
HU-03	Un refresh token válido genera un nuevo access token y otro refresh token; el anterior queda revocado.
HU-04	Al cerrar sesión, el refresh token se revoca y el frontend elimina los tokens locales.
HU-05	GET /api/auth/me requiere Bearer JWT válido y devuelve datos públicos del usuario.
HU-06	PUT /api/auth/me permite actualizar nombre y/o teléfono con validación.
HU-07	DELETE /api/auth/me cambia estado a inactivo y revoca los refresh tokens del usuario.
HU-09	El usuario autenticado puede acceder al dashboard; sin sesión se redirige a /login.
HU-10	El mapa carga Leaflet/OpenStreetMap y muestra marcadores de demostración.
HU-19	Un enlace de recuperación expira a los 15 minutos y solo puede usarse una vez (PASSWORD_RESET_TOKEN.usado).
HU-20	El recibo escaneado queda en estado 'pendiente' hasta que el usuario confirme el gasto extraído; solo entonces se crea el GASTO asociado.

5. Esquema de base de datos

La base de datos utiliza PostgreSQL. Esta sección integra dos niveles: (a) el esquema efectivamente implementado y en producción (USUARIO y REFRESH_TOKEN) y (b) el diseño de datos completo propuesto para los 7 módulos funcionales del sistema, que amplía el modelo actual con las entidades de grupos, gastos, balances, movimientos, notificaciones, recuperación de contraseña y escaneo de recibos.

5.1. Modelo relacional implementado (versión actual)

```
USUARIO (PK id, nombre, email UNIQUE, password_hash, telefono, fecha_registro, estado) 1—N
REFRESH_TOKEN (PK id, FK usuario_id → USUARIO.id, token UNIQUE, fecha_creacion,
fecha_expiracion, revocado)
```

Relación: un USUARIO puede tener cero o muchos REFRESH_TOKEN. Cada REFRESH_TOKEN pertenece a un único USUARIO.

5.1.1. Tabla USUARIO (implementada)

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador único.
nombre	VARCHAR(120)	NOT NULL	Nombre del usuario.
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo usado para autenticación.
password_hash	VARCHAR(255)	NOT NULL	Hash Bcrypt de la contraseña.
telefono	VARCHAR(20)	NULL	Teléfono opcional.
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT NOW()	Fecha de creación.

Campo	Tipo	Restricciones	Descripción
estado	VARCHAR(20)	NOT NULL, CHECK	activo, inactivo o bloqueado.

5.1.2. Tabla REFRESH_TOKEN (implementada)

Campo	Tipo	Restricciones	Descripción
id	BIGSERIAL	PK	Identificador del registro.
usuario_id	BIGINT	FK, NOT NULL	Referencia a USUARIO.id.
token	VARCHAR(500)	UNIQUE, NOT NULL	Token opaco de renovación.
fecha_creacion	TIMESTAMP	NOT NULL	Momento de emisión.
fecha_expiracion	TIMESTAMP	NOT NULL	Fecha de vencimiento.
revocado	BOOLEAN	NOT NULL, DEFAULT FALSE	Indica si el token fue revocado.

Discrepancia identificada entre las dos fuentes (a resolver con el equipo de desarrollo): el diseño ampliado (documento 2) propone almacenar el refresh token como token_hash VARCHAR(255) en vez del token VARCHAR(500) en texto plano actualmente implementado. Se recomienda evaluar migrar a almacenar el hash del token en vez del token opaco directo, como mejora de seguridad, sin cambiar el comportamiento funcional.

5.2. Modelo entidad-relación completo (diseño propuesto para los 7 módulos)

Relaciones principales entre entidades del diseño completo:

- Un USUARIO puede crear varios GRUPO (1:N).
- Un USUARIO participa en varios GRUPO a través de MIEMBRO_GRUPO (relación N:M resuelta con tabla intermedia, incluye rol: administrador/miembro).
- Un GRUPO contiene varios GASTO; cada GASTO tiene un único usuario pagador.
- Cada GASTO se descompone en varias filas de PARTICIPACION_GASTO (una por cada usuario que debe pagar su parte) — aquí vive el "cálculo de participaciones".
- BALANCE almacena el saldo neto consolidado entre pares de usuarios dentro de un grupo (quién le debe a quién), recalculado por el motor de cálculo cada vez que cambia un gasto o un pago.
- MOVIMIENTO es un log de trazabilidad (auditoría) de toda acción financiera relevante, ligada opcionalmente a un gasto.
- RECIBO_ESCANEADO guarda el resultado crudo del pipeline de OCR/Deep Learning antes/después de la confirmación del usuario, y se enlaza 1:1 (opcional) con el GASTO que finalmente se registra.
- NOTIFICACION y REFRESH_TOKEN son tablas de soporte para el panel de usuario y la autenticación, respectivamente.
- PASSWORD_RESET_TOKEN almacena el token de un solo uso (hasheado) para el flujo de recuperación de contraseña, con expiración de 15 minutos.

FT. GROUP — Diagrama Entidad-Relación (vista general)

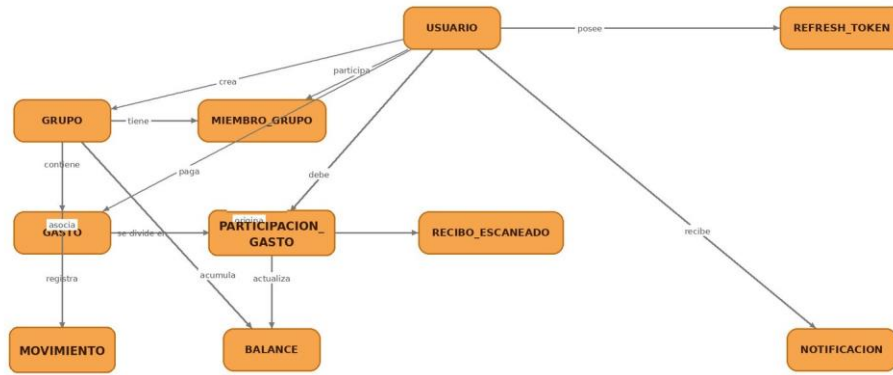


Figura 3. Diagrama entidad-relación general del diseño completo de FT. GROUP.

5.3. Diccionario de datos completo (DDL lógico)

5.3.1. GRUPO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
nombre	VARCHAR(120)	NOT NULL
descripcion	TEXT	NULL
creado_por	BIGINT	FK → USUARIO(id), NOT NULL
fecha_creacion	DATETIME	NOT NULL, DEFAULT now()
estado	ENUM('activo','archivado')	NOT NULL, DEFAULT 'activo'

5.3.2. MIEMBRO_GRUPO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
grupo_id	BIGINT	FK → GRUPO(id), NOT NULL
usuario_id	BIGINT	FK → USUARIO(id), NOT NULL
rol	ENUM('administrador','miembro')	NOT NULL, DEFAULT 'miembro'
fecha_union	DATETIME	NOT NULL, DEFAULT now()
estado	ENUM('activo','expulsado','salido')	NOT NULL, DEFAULT 'activo'
—	—	UNIQUE(grupo_id, usuario_id)

5.3.3. GASTO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
grupo_id	BIGINT	FK → GRUPO(id), NOT NULL
pagador_id	BIGINT	FK → USUARIO(id), NOT NULL
descripcion	VARCHAR(255)	NOT NULL
monto_total	DECIMAL(12,2)	NOT NULL, CHECK (monto_total > 0)

Campo	Tipo	Restricciones
fecha_gasto	DATE	NOT NULL
categoria	VARCHAR(60)	NOT NULL
tipo_division	ENUM('equitativa','personalizada')	NOT NULL
comprobante_url	VARCHAR(500)	NULL
origen_registro	ENUM('manual','ocr')	NOT NULL, DEFAULT 'manual'
fecha_registro	DATETIME	NOT NULL, DEFAULT now()

5.3.4. PARTICIPACION_GASTO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
gasto_id	BIGINT	FK → GASTO(id), NOT NULL, ON DELETE CASCADE
usuario_id	BIGINT	FK → USUARIO(id), NOT NULL
monto_asignado	DECIMAL(12,2)	NOT NULL, CHECK (monto_asignado >= 0)
estado_pago	ENUM('pendiente','pagado')	NOT NULL, DEFAULT 'pendiente'
fecha_pago	DATETIME	NULL
—	—	UNIQUE(gasto_id, usuario_id)

Regla de integridad de aplicación: SUM(monto_asignado) de todas las filas de un gasto_id debe ser igual a GASTO.monto_total (validado en el motor de cálculo, dentro de una transacción ACID).

5.3.5. BALANCE (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
grupo_id	BIGINT	FK → GRUPO(id), NOT NULL
deudor_id	BIGINT	FK → USUARIO(id), NOT NULL
acreedor_id	BIGINT	FK → USUARIO(id), NOT NULL
monto	DECIMAL(12,2)	NOT NULL, CHECK (monto >= 0)
fecha_actualizacion	DATETIME	NOT NULL, DEFAULT now()
—	—	UNIQUE(grupo_id, deudor_id, acreedor_id)

5.3.6. MOVIMIENTO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
usuario_id	BIGINT	FK → USUARIO(id), NOT NULL
grupo_id	BIGINT	FK → GRUPO(id), NOT NULL
gasto_id	BIGINT	FK → GASTO(id), NULL

Campo	Tipo	Restricciones
tipo_movimiento	ENUM('gasto_creado','gasto_editado','pago_registrado','miembro_agregado','miembro_expulsado')	NOT NULL
descripcion	TEXT	NULL
fecha	DATETIME	NOT NULL, DEFAULT now()

5.3.7. RECIBO_ESCANEADO (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
gasto_id	BIGINT	FK → GASTO(id), NULL (nulo hasta confirmación)
imagen_url	VARCHAR(500)	NOT NULL
monto_extraido	DECIMAL(12,2)	NULL
fecha_extraida	DATE	NULL
comercio_extraido	VARCHAR(150)	NULL
categoria_sugerida	VARCHAR(60)	NULL
confianza_ocr	DECIMAL(5,2)	NULL (% de confianza del modelo)
estado_validacion	ENUM('pendiente','confirmado','rechazado')	NOT NULL, DEFAULT 'pendiente'
fecha_procesamiento	DATETIME	NOT NULL, DEFAULT now()

5.3.8. NOTIFICACION (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
usuario_id	BIGINT	FK → USUARIO(id), NOT NULL
tipo	ENUM('nuevo_gasto','pago_recibido','recordatorio','invitation_grupo')	NOT NULL
mensaje	TEXT	NOT NULL
leido	BOOLEAN	NOT NULL, DEFAULT false
fecha_creacion	DATETIME	NOT NULL, DEFAULT now()

5.3.9. PASSWORD_RESET_TOKEN (nueva)

Campo	Tipo	Restricciones
id	BIGINT	PK
usuario_id	BIGINT	FK → USUARIO(id), NOT NULL
token_hash	VARCHAR(255)	NOT NULL, UNIQUE
expira_en	DATETIME	NOT NULL
usado	BOOLEAN	NOT NULL, DEFAULT false

Campo	Tipo	Restricciones
creado_en	DATETIME	NOT NULL, DEFAULT now()

5.4. Relaciones e índices

Índices y relaciones ya implementados:

- FK refresh_token.usuario_id → usuario.id con ON DELETE CASCADE.
- Índice único idx_usuario_email para búsquedas y unicidad del correo.
- Índice idx_refresh_token_usuario para localizar tokens por usuario.
- Índice idx_refresh_token_token para localizar rápidamente un refresh token.
- El borrado lógico del usuario se realiza cambiando estado a inactivo; por tanto, normalmente el registro permanece en la base de datos.

Índices recomendados para el diseño completo (eficiencia — ISO/IEC 25010):

- USUARIO(email) — único, usado en login. (ya implementado)
- MIEMBRO_GRUPO(grupo_id, usuario_id) — único, consultas de membresía.
- GASTO(grupo_id, fecha_gasto) — filtros de historial por grupo y fecha.
- PARTICIPACION_GASTO(usuario_id, estado_pago) — cálculo rápido de deudas pendientes por usuario.
- BALANCE(grupo_id) — carga del panel de usuario/dashboard.
- MOVIMIENTO(grupo_id, fecha) — reportes e historial paginado.
- RECIBO_ESCANEADO(estado_validacion) — cola de recibos pendientes de confirmar.

6. Documentación técnica

6.1. Tecnologías

Tecnología	Uso
Node.js	Entorno de ejecución del backend.
TypeScript	Tipado y desarrollo del backend.
Express 4	Framework HTTP y API REST.
PostgreSQL 16	Base de datos relacional.
Docker Compose	Ejecución local de PostgreSQL.
Bcrypt	Hash y comparación de contraseñas.
JSON Web Token	Access tokens.
Zod	Validación de entradas.
Helmet	Cabeceras HTTP de seguridad.
React 18	Interfaz de usuario.
Vite 5	Servidor y build del frontend.
React Router 6	Navegación y rutas protegidas.
Leaflet / React-Leaflet	Mapa funcional.
OpenStreetMap	Capa de mapas.

Tecnología	Uso
Jest	Pruebas unitarias del backend.

6.2. Estructura de carpetas

```

ft-group-completo/ ├── ft-group-backend/ ├── ft-group-backend/ ├── src/ |
| ├── domain/ | | ├── application/ | | ├── infrastructure/ | | |
db/migrations/ | | ├── tests/unit/ | | ├── dist/ | | ├── docker-compose.yml |
| ├── package.json | | ├── ft-group-frontend/ | | ├── src/ | | ├── components/ | | |
pages/ | | | api.js | | | App.jsx | | | main.jsx | | | vite.config.js
| | | package.json

```

6.3. API REST (implementada)

Método	Ruta	Auth	Función	Respuesta principal
GET	/health	No	Verificar estado del servicio.	200 OK
POST	/api/auth/registro	No	Registrar usuario.	201 Created
POST	/api/auth/login	No	Iniciar sesión.	200 OK
POST	/api/auth/refresh	No	Renovar tokens.	200 OK
POST	/api/auth/logout	No*	Revocar refresh token.	204 No Content
GET	/api/auth/me	JWT	Consultar perfil.	200 OK
PUT	/api/auth/me	JWT	Actualizar perfil.	200 OK
DELETE	/api/auth/me	JWT	Desactivar cuenta.	200 OK
POST	/api/auth/me/activar	JWT	Activar cuenta.	200 OK

* Logout valida el refresh token recibido en el cuerpo de la petición. El access token no es necesario para esa ruta.

6.4. Rutas del frontend

Ruta	Pantalla	Protección
/login	Inicio de sesión	Pública
/registro	Registro de usuario	Pública
/dashboard	Dashboard y mapa	Protegida
/perfil	Perfil del usuario	Protegida
*	Redirección según sesión	Automática

6.5. Variables de entorno

Backend (.env):

```

NODE_ENV=development PORT=3000 DATABASE_URL= DATABASE_SSL=false DB_HOST=localhost
DB_PORT=5432 DB_USER=postgres DB_PASSWORD=postgres DB_NAME=ft_group JWT_SECRET=cambia-este-
secreto JWT_EXPIRATION=15m CORS_ORIGIN=http://localhost:5173

```

Frontend (.env):

```
VITE_API_URL=http://localhost:3000/api
```

En producción se debe cambiar JWT_SECRET por un secreto robusto y no reutilizar las credenciales de desarrollo incluidas en Docker Compose.

6.6. Instalación y ejecución

Backend:

```
cd ft-group-backend/ft-group-backend npm install docker compose up -d npm run migrate:dev npm run dev
```

El backend queda disponible, por defecto, en <http://localhost:3000>.

Frontend, en una segunda terminal:

```
cd ft-group-frontend npm install npm run dev
```

El frontend queda disponible, por defecto, en <http://localhost:5173>.

6.7. Pruebas

El repositorio incluye pruebas unitarias con Jest para los casos de uso de registro e inicio de sesión. Las pruebas utilizan mocks de los puertos de salida, por lo que no necesitan una base de datos real para comprobar esas reglas.

```
npm test npm run lint npm run build
```

Estas instrucciones ejecutan las pruebas, verifican el tipado y generan el build del backend. El documento describe las pruebas existentes en el código; su ejecución debe realizarse en el entorno local después de instalar las dependencias.

7. Estado actual, limitaciones y trabajo futuro

Área	Estado actual	Siguiente trabajo
Backend	Funcional para usuarios y autenticación.	Agregar módulo completo de grupos y gastos.
CRUD de usuarios	Registro, lectura de perfil, actualización y desactivación propia.	Agregar endpoints administrativos de listado, edición por ID y gestión de estado.
JWT	Implementado.	Mantener expiraciones y secretos adecuados en producción.
Refresh token	Implementado con rotación y revocación.	Migrarlo a cookie HttpOnly/Secure/SameSite; evaluar almacenar como token_hash.
Recuperación de contraseña	No implementado.	Implementar flujo con PASSWORD_RESET_TOKEN (ya diseñado).
Frontend	Login, registro, dashboard, perfil y mapa.	Conectar indicadores y mapa a datos reales.
Dashboard	Datos básicos de demostración.	Conectar grupos, gastos y pendientes a API.
Mapa	Leaflet con marcadores de ejemplo.	Persistir ubicaciones y asociarlas a actividades reales.
Base de datos	USUARIO y REFRESH_TOKEN.	Agregar entidades de grupos, integrantes, gastos, pagos y categorías (ya diseñadas: GRUPO, MIEMBRO_GRUPO, GASTO, PARTICIPACION_GASTO, BALANCE,

Área	Estado actual	Siguiente trabajo
		MOVIMIENTO).
Escaneo de recibos (IA)	No implementado.	Implementar microservicio de OCR/Deep Learning y tabla RECIBO_ESCANEADO (ya diseñada).
Notificaciones	No implementado.	Implementar módulo de notificaciones con la tabla NOTIFICACION (ya diseñada).
OAuth	No implementado.	Evaluar proveedor externo si el alcance del proyecto lo requiere.
Pruebas	Pruebas unitarias de casos principales.	Agregar pruebas de integración y endpoints HTTP; alcanzar cobertura objetivo > 80 %.

8. Notas y limitaciones de la fusión de documentos

Este documento integra dos fuentes de distinta naturaleza y alcance temporal, por lo que se documentan aquí los criterios de fusión aplicados:

- El documento fuente 1 ("Documento de Arquitectura, Backlog Inicial, Esquema de BD y Documentación Técnica", v1.0) describe fielmente la implementación real entregada del proyecto: únicamente el módulo de usuarios y autenticación, con dos tablas (USUARIO, REFRESH_TOKEN).
- El documento fuente 2 ("FT_GROUP Esquema de BD y Documentación Técnica", v2.1) es un diseño técnico propuesto e inferido a partir del anteproyecto académico de la Universidad de San Buenaventura, que cubre los 7 módulos funcionales completos del sistema, incluyendo entidades y arquitectura que aún no han sido implementadas.
- El diccionario de datos de la sección 5.3 fue elaborado originalmente a partir de una propuesta que planteaba una arquitectura Microkernel; su nomenclatura ya fue unificada con el documento oficial de arquitectura (PARTICIPACION_GASTO y MOVIMIENTO), y se agregó la entidad PASSWORD_RESET_TOKEN para reflejar el flujo de recuperación de contraseña previsto.
- La sección de arquitectura se actualizó íntegramente con base en el documento "Definición de Arquitectura del Sistema", que reemplaza el enfoque Microkernel por la Arquitectura Hexagonal (Ports & Adapters) como estilo arquitectónico oficial de FT. GROUP — estilo que coincide con el efectivamente implementado en el backend actual.
- Se identificó y documentó una discrepancia entre ambas fuentes respecto al campo de almacenamiento del refresh token (token en texto plano vs. token_hash), señalada como nota de mejora en la sección 5.1.2.

Se recomienda validar este documento integrado con el equipo de desarrollo antes de usarlo como base definitiva de implementación de los módulos pendientes (por ejemplo, definir si se requieren tablas adicionales para "reportes exportados" o configuración de categorías personalizadas por grupo).

9. Conclusiones

La versión actual de FT. GROUP establece una base técnica funcional para continuar el desarrollo del sistema. El backend está organizado mediante arquitectura hexagonal y cuenta con un módulo de usuarios, autenticación JWT, refresh tokens, persistencia PostgreSQL, validación de entradas y medidas básicas de seguridad.

El frontend inicial permite recorrer el flujo principal de registro, inicio de sesión, acceso protegido, consulta y actualización del perfil y visualización de un dashboard con mapa. Esto permite validar la integración entre frontend y backend antes de incorporar la lógica completa de grupos y gastos compartidos.

La fusión de ambos documentos deja disponible, en un único lugar, tanto el estado real del proyecto como el diseño completo de datos y arquitectura para los módulos restantes (grupos, gastos, motor de cálculo, historial, notificaciones y escaneo inteligente de recibos), unificando nomenclatura y sirviendo como hoja de ruta técnica coherente para las siguientes iteraciones.

Como siguientes pasos técnicos se recomienda completar los módulos funcionales del dominio siguiendo el mismo patrón hexagonal ya validado, conectar el dashboard a datos reales, fortalecer la gestión de sesiones mediante cookies seguras, implementar la recuperación de contraseña y el módulo de notificaciones, y ampliar las pruebas automatizadas e de integración hasta alcanzar la cobertura objetivo.